

Guía paso a paso: diseño y fragmentación de una base de datos distribuida

Fragmentación horizontal, vertical y mixta sobre PostgreSQL en contenedores

Aplicaciones Distribuidas · Séptimo semestre · ISR-701

Universidad Técnica Estatal de Quevedo · Facultad de Ciencias de la Computación

Carrera: Ingeniería de Software

Asignatura: Aplicaciones Distribuidas (ISR-701)

Documento: Guía de práctica — Fragmentación de BD

Semana de entrega: Semanas 8–10

Período: Regular 2026-2027

Modalidad: Grupal (equipos PFC de 3-4 integrantes)

Docente responsable: Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Versión: v1.0

1. ¿Por qué necesitas esta práctica?

Imagina que administras la biblioteca de la universidad. Al principio hay un solo edificio con todos los libros, y todo funciona bien. Pero un día se abre una sede en Babahoyo y otra en Ventanas, y la biblioteca central se llena de gente que espera. La solución obvia no es construir un edificio más grande, sino dividir los libros: los libros que casi solo consultan en Babahoyo se llevan a Babahoyo, y los de Ventanas a Ventanas. Esa idea sencilla, aplicada a las bases de datos, se llama *fragmentación*, y es el tema de la práctica que vas a realizar.

Al terminar esta práctica sabrás por qué se fragmenta una base de datos, distinguirás la fragmentación horizontal de la vertical, harás las dos con SQL puro sobre PostgreSQL, y comprobarás que tu diseño es correcto. No necesitas experiencia previa con bases de datos distribuidas; te explicaremos cada palabra la primera vez que aparezca.

Costo cero. Todo lo que usarás en esta guía es libre y gratuito: Docker Desktop, PostgreSQL 16 y `psql`. No debes crear cuentas de pago ni ingresar tarjetas. Con una computadora con 4 GB de RAM libres es suficiente.

2. Vocabulario mínimo (léelo antes de continuar)

Los términos que siguen aparecerán durante toda la guía. Cada uno se define en una sola oración para que no tengas que salir a buscar en otro lado.

- **Base de datos (BD):** conjunto organizado de datos guardado en disco y accedido a través de un programa que se llama *sistema gestor de base de datos* o SGBD.
- **SGBD:** el programa que administra la BD; en esta práctica el SGBD será PostgreSQL.
- **Tabla o relación:** colección de filas con la misma estructura, como una hoja de cálculo. La palabra formal en la teoría es *relación* [1].
- **Fila o tupla:** un registro dentro de una tabla; en teoría se le llama *tupla*.
- **Columna o atributo:** una característica de la tabla, como **nombre** o **fecha**.
- **Clave primaria (PK):** atributo (o combinación) que identifica cada fila de forma única. En `clientes` podría ser `cliente_id`.
- **Clave foránea (FK):** atributo de una tabla que apunta a la clave primaria de otra; sirve para relacionarlas.
- **Esquema (schema):** el “plano” de la BD: qué tablas hay y qué columnas tienen.
- **Nodo:** una máquina (o un contenedor Docker) que corre una copia del SGBD.
- **Clúster:** un grupo de nodos que trabajan juntos como si fueran uno solo.
- **Fragmentación:** dividir una tabla en pedazos más pequeños llamados *fragmentos*, que se guardan en nodos distintos [1, 2].

- **Fragmentación horizontal:** se corta la tabla por *filas*. Cada fragmento tiene las mismas columnas pero distintas filas.
- **Fragmentación vertical:** se corta la tabla por *columnas*. Cada fragmento tiene las mismas filas pero distintas columnas [3].
- **Fragmentación mixta:** se aplican las dos anteriores sobre la misma tabla.
- **Docker:** programa que empaqueta software en *contenedores*, para que corra igual en cualquier computadora.
- **Docker Compose:** archivo YAML donde declaras varios contenedores y cómo se relacionan.
- **Vista (*view*):** consulta SQL guardada con un nombre; al llamarla, se comporta como si fuera una tabla.

3. Fundamento teórico mínimo

3.1. ¿Por qué fragmentar?

Fragmentar tiene tres beneficios que la literatura clásica de bases de datos distribuidas reconoce desde hace más de cuatro décadas [2, 1]: (i) rendimiento, porque cada nodo solo carga con una parte de los datos; (ii) localidad, porque los datos pueden vivir cerca de quien más los usa; y (iii) escalabilidad, porque cuando la BD crece, se añaden nodos en lugar de un servidor más grande.

A cambio, el sistema se complica: hay que decidir cómo cortar, cómo reunir los pedazos cuando alguien hace una consulta general, y cómo evitar duplicados o pérdidas. Esta guía te enseña exactamente eso.

3.2. Tres condiciones de corrección

Sin importar cómo cortes tu tabla, tu diseño debe cumplir tres reglas irrenunciables [1]. Estas tres reglas son la vara con la que se mide si una fragmentación es correcta.

Reglas que TODA fragmentación debe cumplir

1. **Complejitud.** Ningún dato se pierde. Cada fila (o cada columna, según el caso) de la tabla original debe estar en al menos un fragmento.
2. **Reconstrucción.** La tabla original se puede armar de nuevo a partir de sus fragmentos, sin ambigüedad. En la práctica esto se hace con UNION para el corte horizontal y con JOIN para el vertical.
3. **Disjunción.** Ningún dato está repetido en más de un fragmento (excepto la clave primaria, que sí se repite en la fragmentación vertical para poder reunir las columnas después).

3.3. Cómo se ve cada tipo de corte

La Figura 1 ilustra los dos tipos de fragmentación básicos. Al leerla, observa que la fragmentación horizontal mantiene todas las columnas y reparte las filas, mientras que la vertical mantiene todas las filas y reparte las columnas.

3.4. Nuestro escenario de ejemplo: Cafetería UTEQ

Trabajaremos con una cafetería universitaria pequeña que tiene una base de datos con tres tablas: **clientes** (estudiantes registrados), **productos** (café, sánduches, jugos) y **pedidos** (una venta realizada). La cafetería empezó con una sola sede en el campus principal, pero abrió sucursales en dos edificios más. Ese crecimiento es la excusa para fragmentar por sede y por tipo de dato.

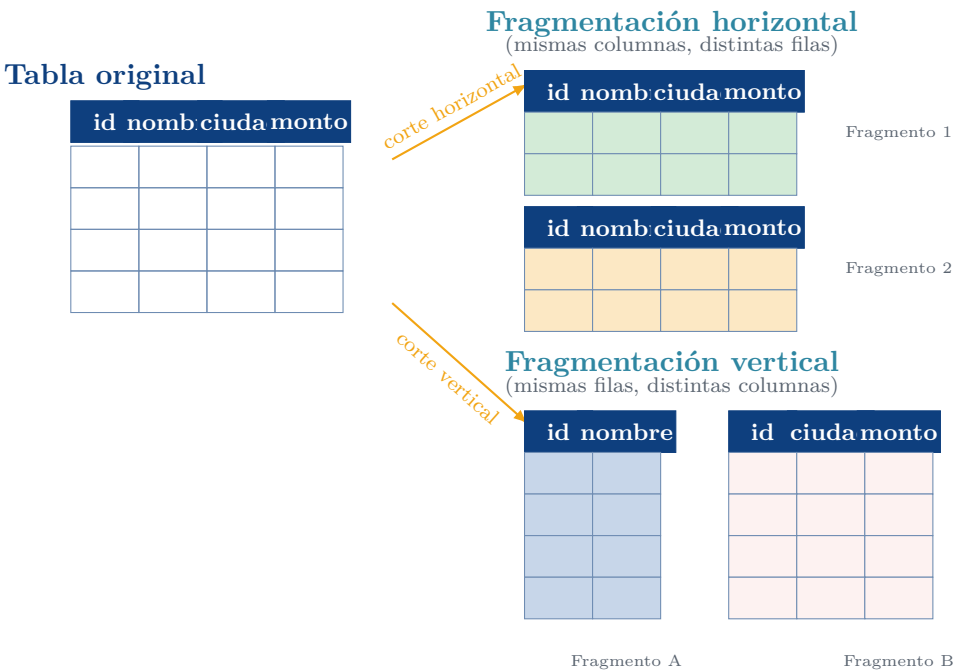


Figura 1: Fragmentación horizontal frente a fragmentación vertical. La columna `id` se repite en los fragmentos verticales para permitir reunir las columnas mediante JOIN.

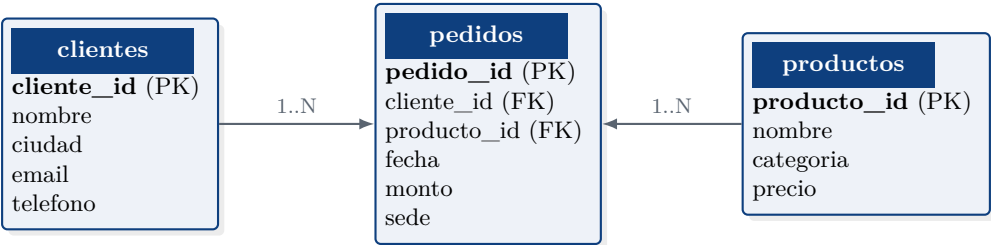


Figura 2: Modelo entidad-relación del ejemplo. Un cliente puede tener muchos pedidos; cada pedido involucra un producto.

4. Preparación del entorno de trabajo

Antes de empezar a escribir SQL necesitas tener corriendo tres nodos PostgreSQL. Cada nodo será un contenedor Docker independiente, y cada uno alojará un fragmento de la base de datos.

4.1. Requisitos previos

- Computadora con al menos 4 GB de RAM libres y 5 GB de espacio en disco.
- Docker Desktop instalado (Windows, macOS o Linux). Descárgalo de <https://www.docker.com/products/docker-desktop/>. Es gratuito para uso educativo.
- Un editor de código como Visual Studio Code (libre) o cualquier editor de texto.
- Terminal para escribir comandos. En Windows usa PowerShell o CMD; en macOS y Linux, la terminal por defecto.

4.2. Verifica que Docker funciona

Abre una terminal y ejecuta el comando del Listado 1. Si Docker responde con su versión, ya estás listo; si te aparece un error, revisa que Docker Desktop esté iniciado (verás su ícono en la barra de tareas).

```
1 docker --version
2 docker compose version
3
```

Listing 1: Verificación de que Docker está corriendo.

4.3. Crea la carpeta del proyecto

Crea una carpeta nueva en un sitio cómodo, por ejemplo en tu Escritorio. Dentro tendrás un archivo `docker-compose.yml` y una carpeta `sql/` para los scripts SQL.

```
1 practica-fragmentacion/
2   docker-compose.yml
3   sql/
4     01_esquema_central.sql
5     02_datos.sql
6     03_fragmentacion_horizontal.sql
7     04_fragmentacion_vertical.sql
8     05_fragmentacion_mixta.sql
9     06_vistas_globales.sql
10    07_verificacion.sql
11
```

Listing 2: Estructura de carpetas del proyecto de la práctica.

4.4. Levanta los tres nodos PostgreSQL

Copia el contenido del Listado 3 dentro de `docker-compose.yml`. Este archivo declara tres contenedores PostgreSQL, uno por cada sede de la cafetería: campus, Babahoyo y Ventanas.

```
1 services:
2   pg-campus:
3     image: postgres:16-alpine
4     container_name: pg-campus
5     environment:
6       POSTGRES_DB: cafeteria
7       POSTGRES_USER: admin
8       POSTGRES_PASSWORD: admin123
9     ports: ["5433:5432"]
10    volumes:
11      - pg-campus-data:/var/lib/postgresql/data
12
```

```
13 pg-babahoyo:
14   image: postgres:16-alpine
15   container_name: pg-babahoyo
16   environment:
17     POSTGRES_DB: cafeteria
18     POSTGRES_USER: admin
19     POSTGRES_PASSWORD: admin123
20   ports: ["5434:5432"]
21   volumes:
22     - pg-babahoyo-data:/var/lib/postgresql/data
23
24 pg-ventanas:
25   image: postgres:16-alpine
26   container_name: pg-ventanas
27   environment:
28     POSTGRES_DB: cafeteria
29     POSTGRES_USER: admin
30     POSTGRES_PASSWORD: admin123
31   ports: ["5435:5432"]
32   volumes:
33     - pg-ventanas-data:/var/lib/postgresql/data
34
35 volumes:
36   pg-campus-data:
37   pg-babahoyo-data:
38   pg-ventanas-data:
39
```

Listing 3: docker-compose.yml con tres nodos PostgreSQL, uno por sede.

Ahora, en la misma carpeta del archivo, ejecuta el comando del Listado 4. La primera vez tardará uno o dos minutos porque Docker descarga la imagen de PostgreSQL.

```
1 docker compose up -d
2 docker compose ps
3
```

Listing 4: Levantar los tres nodos en segundo plano.

Deberías ver los tres contenedores en estado **running**. Si algo falla, mira el mensaje: los errores más comunes son que el puerto ya está ocupado (cámbialo por otro) o que Docker no tiene permiso (reinicia Docker Desktop).

4.5. Conéctate a cada nodo

Para hablar con PostgreSQL usaremos su cliente estándar **psql**, que ya viene dentro del contenedor. El Listado 5 muestra cómo abrir una sesión SQL en el nodo de campus. Cambia **pg-campus** por **pg-babahoyo** o **pg-ventanas** para conectarte a los otros.

```
1 docker exec -it pg-campus psql -U admin -d cafeteria
2
```

Listing 5: Abrir una sesión psql dentro de un contenedor.

Sal de **psql** escribiendo **\q** y presionando **Enter**. Con esto, el entorno queda listo y podemos empezar el diseño.

La Figura 3 muestra cómo queda montada la infraestructura una vez levantada.

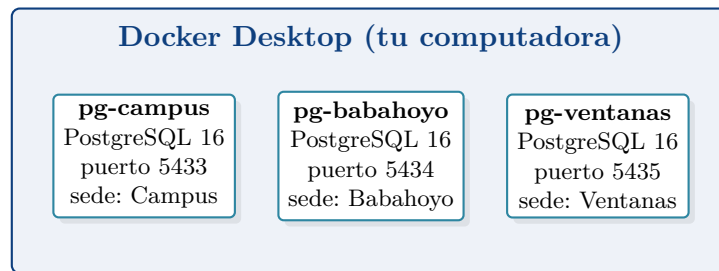


Figura 3: Tres nodos PostgreSQL independientes corriendo como contenedores en la misma máquina. Cada nodo actúa como una sede de la cafetería.

5. Paso 1: diseño del esquema centralizado

Antes de fragmentar tienes que saber qué vas a fragmentar. Vamos a empezar creando la BD completa en un solo nodo (pg-campus) para tener una referencia. El Listado 6 contiene el script; guárdalo en `sql/01_esquema_central.sql`.

```

1 CREATE TABLE clientes (
2     cliente_id SERIAL PRIMARY KEY,
3     nombre     VARCHAR(80) NOT NULL,
4     ciudad     VARCHAR(40) NOT NULL,
5     email      VARCHAR(120) NOT NULL,
6     telefono   VARCHAR(20)
7 );
8
9 CREATE TABLE productos (
10     producto_id SERIAL PRIMARY KEY,
11     nombre      VARCHAR(80) NOT NULL,
12     categoria   VARCHAR(30) NOT NULL,
13     precio      NUMERIC(6,2) NOT NULL CHECK (precio >= 0)
14 );
15
16 CREATE TABLE pedidos (
17     pedido_id SERIAL PRIMARY KEY,
18     cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id),
19     producto_id INTEGER NOT NULL REFERENCES productos(producto_id),
20     fecha      DATE NOT NULL,
21     monto      NUMERIC(8,2) NOT NULL CHECK (monto >= 0),
22     sede       VARCHAR(20) NOT NULL
23 );
24

```

Listing 6: `01_esquema_central.sql` — esquema completo antes de fragmentar.

Aplica el script al nodo de campus con el comando del Listado 7. Este comando pasa el archivo SQL al contenedor y lo ejecuta. Si no ves errores, tu esquema central ya está creado.

```

1 docker exec -i pg-campus psql -U admin -d cafeteria < sql/01_esquema_central.sql
2

```

Listing 7: Aplicar el esquema al nodo de campus.

6. Paso 2: cargar datos de prueba

Necesitamos datos para ver cómo se cortan. El Listado 8 inserta un puñado de filas de ejemplo. Guárdalo como `sql/02_datos.sql` y aplícalo con el mismo comando anterior cambiando el nombre del archivo.

```

1 INSERT INTO clientes (nombre, ciudad, email, telefono) VALUES
2 ('Maria Alvarado', 'Quevedo', 'maria@uteq.edu.ec', '0991111111'),
3 ('Luis Cedeno', 'Babahoyo', 'luis@uteq.edu.ec', '0992222222'),
4 ('Ana Vera', 'Quevedo', 'ana@uteq.edu.ec', '0993333333'),

```

```

5 ('Jose Mendoza', 'Ventanas', 'jose@uteq.edu.ec', '0994444444'),
6 ('Carla Zambrano', 'Ventanas', 'carla@uteq.edu.ec', '0995555555'),
7 ('Pedro Suarez', 'Babahoyo', 'pedro@uteq.edu.ec', '0996666666');
8
9 INSERT INTO productos (nombre, categoria, precio) VALUES
10 ('Cafe negro', 'bebida', 0.75),
11 ('Cafe con leche', 'bebida', 1.00),
12 ('Sanduche pollo', 'comida', 2.50),
13 ('Jugo natural', 'bebida', 1.25),
14 ('Empanada', 'comida', 1.00);
15
16 INSERT INTO pedidos (cliente_id, producto_id, fecha, monto, sede) VALUES
17 (1, 1, '2026-03-01', 0.75, 'Campus'),
18 (2, 3, '2026-03-01', 2.50, 'Babahoyo'),
19 (3, 2, '2026-03-02', 1.00, 'Campus'),
20 (4, 4, '2026-03-02', 1.25, 'Ventanas'),
21 (5, 5, '2026-03-03', 1.00, 'Ventanas'),
22 (6, 1, '2026-03-03', 0.75, 'Babahoyo'),
23 (1, 3, '2026-03-04', 2.50, 'Campus'),
24 (2, 2, '2026-03-04', 1.00, 'Babahoyo');
25

```

Listing 8: 02_datos.sql — inserción de datos de prueba.

7. Paso 3: fragmentación horizontal

7.1. Idea intuitiva

Vamos a decir que cada pedido se atiende en la sede donde se generó, y por lo tanto *cada nodo guardará solo los pedidos de su sede*. La tabla `pedidos` tiene la columna `sede`, así que el criterio es directo. En el lenguaje del álgebra relacional esto es una operación de *selección* [1]:

$$\text{pedidos_campus} = \sigma_{\text{sede}='Campus'}(\text{pedidos}) \quad (1)$$

$$\text{pedidos_babahoyo} = \sigma_{\text{sede}='Babahoyo'}(\text{pedidos}) \quad (2)$$

$$\text{pedidos_ventanas} = \sigma_{\text{sede}='Ventanas'}(\text{pedidos}) \quad (3)$$

donde σ es el símbolo estándar para “filtra las filas que cumplen la condición”. Los tres predicados (Campus, Babahoyo, Ventanas) son mutuamente excluyentes (una fila no puede pertenecer a dos sedes) y colectivamente exhaustivos (toda fila cae en exactamente una sede), lo que garantiza las tres condiciones de corrección.

La Figura 4 lo muestra visualmente.

7.2. Procedimiento

En cada nodo se crea una copia del esquema, pero *solo* con las filas que le corresponden. Guarda el Listado 9 como `sql/03_fragmentacion_horizontal.sql`.

```

1 -- Este script se aplica en cada nodo, pero solo se insertan
2 -- las filas cuya columna sede coincide con la sede del nodo.
3
4 CREATE TABLE pedidos (
5     pedido_id    INTEGER    PRIMARY KEY,
6     cliente_id   INTEGER    NOT NULL,
7     producto_id  INTEGER    NOT NULL,
8     fecha        DATE       NOT NULL,
9     monto        NUMERIC(8,2) NOT NULL,
10    sede         VARCHAR(20) NOT NULL
11 );
12
13 -- Ejemplo: en pg-babahoyo solo se insertan los pedidos con sede='Babahoyo'.
14 INSERT INTO pedidos VALUES (2, 2, 3, '2026-03-01', 2.50, 'Babahoyo');
15 INSERT INTO pedidos VALUES (6, 6, 1, '2026-03-03', 0.75, 'Babahoyo');
16 INSERT INTO pedidos VALUES (8, 2, 2, '2026-03-04', 1.00, 'Babahoyo');

```

pedidos (tabla completa)				
pedido_id	cliente	fecha	sede	monto
1	1	03-01	Campus	0.75
2	2	03-01	Babahoyo	2.50
3	3	03-02	Campus	1.00
4	4	03-02	Ventanas	1.25
5	5	03-03	Ventanas	1.00
6	6	03-03	Babahoyo	0.75
7	1	03-04	Campus	2.50
8	2	03-04	Babahoyo	1.00

pedidos_campus				
id	cli	fecha	sede	monto
1	1	03-01	Campus	0.75
3	3	03-02	Campus	1.00
7	1	03-04	Campus	2.50

selección por sede →

pedidos_babahoyo				
id	cli	fecha	sede	monto
2	2	03-01	Babahoyo	2.50
6	6	03-03	Babahoyo	0.75
8	2	03-04	Babahoyo	1.00

pedidos_ventanas				
id	cli	fecha	sede	monto
4	4	03-02	Ventanas	1.25
5	5	03-03	Ventanas	1.00

Figura 4: Fragmentación horizontal por sede. Las filas se colorean para que veas hacia qué fragmento va cada una. Las columnas son idénticas en los tres fragmentos.

17

Listing 9: 03_fragmentacion_horizontal.sql — fragmento local por sede.

Repite la misma idea en `pg-campus` y `pg-ventanas`, cambiando los `INSERT` por las filas correspondientes. Para simplificar la práctica, cada equipo puede insertar solo tres o cuatro filas por nodo; lo importante es que ninguna fila esté en dos nodos y que todas estén en alguno.

Alternativa nativa con PostgreSQL. PostgreSQL permite declarar la fragmentación horizontal automáticamente con `PARTITION BY LIST(sede)` [4]. Puedes probarlo en un mismo nodo como ejercicio opcional, pero para esta práctica usaremos tres nodos distintos porque el objetivo es entender la distribución *física* sobre múltiples máquinas.

8. Paso 4: fragmentación vertical

8.1. Idea intuitiva

Ahora atacamos la tabla `clientes`. Nota que algunos atributos casi nunca cambian (`nombre`, `ciudad`) mientras que otros son sensibles y solo los ve el administrador (`email`, `telefono`). Es razonable separarlos: dejamos los **datos públicos** en un fragmento y los **datos de contacto** en otro, más protegido. La operación de álgebra relacional que corresponde es la *proyección* [3]:

$$\text{clientes_publicos} = \pi_{\text{cliente_id}, \text{nombre}, \text{ciudad}}(\text{clientes}) \quad (4)$$

$$\text{clientes_contacto} = \pi_{\text{cliente_id}, \text{email}, \text{telefono}}(\text{clientes}) \quad (5)$$

donde π significa “elige estas columnas”. Fíjate que la clave primaria `cliente_id` aparece en los dos fragmentos. Esa duplicación es intencional: sin ella no podríamos volver a unir las columnas. La Figura 5 lo dibuja para que quede claro.

8.2. Procedimiento

Para esta práctica pondremos `clientes_publicos` en `pg-campus` y `clientes_contacto` en `pg-babahoyo`. Guarda el Listado 10 como `sql/04_fragmentacion_vertical.sql` y aplícalo con `docker exec` en cada nodo correspondiente.

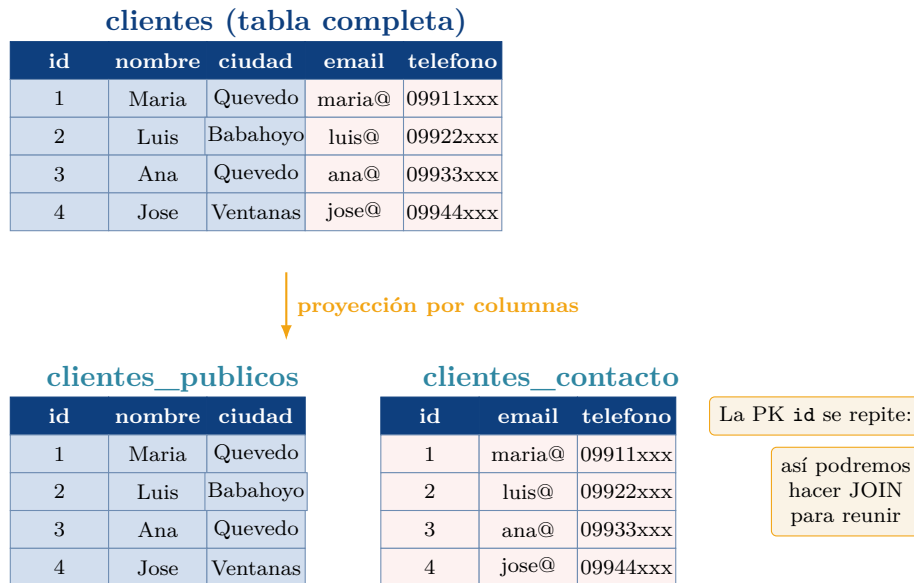


Figura 5: Fragmentación vertical de `clientes`. Todas las filas están en los dos fragmentos, pero cada uno tiene solo un subconjunto de columnas. La clave primaria `cliente_id` se replica intencionalmente.

```

1  -- Se ejecuta en pg-campus (guarda los datos publicos):
2  CREATE TABLE clientes_publicos (
3      cliente_id INTEGER PRIMARY KEY,
4      nombre     VARCHAR(80) NOT NULL,
5      ciudad     VARCHAR(40) NOT NULL
6  );
7
8  INSERT INTO clientes_publicos VALUES
9  (1, 'Maria Alvarado', 'Quevedo'),
10 (2, 'Luis Ceden', 'Babahoyo'),
11 (3, 'Ana Vera', 'Quevedo'),
12 (4, 'Jose Mendoza', 'Ventanas'),
13 (5, 'Carla Zambrano', 'Ventanas'),
14 (6, 'Pedro Suarez', 'Babahoyo');
15
16 -- Se ejecuta en pg-babahoyo (guarda los datos de contacto):
17 CREATE TABLE clientes_contacto (
18     cliente_id INTEGER PRIMARY KEY, -- misma PK que en el otro fragmento
19     email      VARCHAR(120) NOT NULL,
20     telefono   VARCHAR(20)
21 );
22
23 INSERT INTO clientes_contacto VALUES
24 (1, 'maria@uteq.edu.ec', '0991111111'),
25 (2, 'luis@uteq.edu.ec', '0992222222'),
26 (3, 'ana@uteq.edu.ec', '0993333333'),
27 (4, 'jose@uteq.edu.ec', '0994444444'),
28 (5, 'carla@uteq.edu.ec', '0995555555'),
29 (6, 'pedro@uteq.edu.ec', '0996666666');
30

```

Listing 10: 04_fragmentacion_vertical.sql — creación de los dos fragmentos verticales.

9. Paso 5: fragmentación mixta

Un diseño real casi nunca es puro. Aplicaremos las dos técnicas juntas sobre `clientes`: primero cortamos horizontalmente por ciudad y luego cortamos verticalmente por sensibilidad de los datos. El resultado es la Tabla 1.

Guarda el script como `sql/05_fragmentacion_mixta.sql` y aplícalo en cada nodo respetando la Tabla 1. Este es el mejor momento para practicar la lectura crítica del diseño: pregúntate, en cada

Corte 1 (horizontal)	Corte 2 (vertical)	Contenido	Nodo
Ciudad = Quevedo	Datos públicos	cliente_id, nombre, ciudad	pg-campus
Ciudad = Quevedo	Datos contacto	cliente_id, email, telefono	pg-babahoyo
Ciudad = Babahoyo o Ventanas	Datos públicos	cliente_id, nombre, ciudad	pg-ventanas
Ciudad = Babahoyo o Ventanas	Datos contacto	cliente_id, email, telefono	pg-babahoyo

Tabla 1: Fragmentación mixta de **clientes**: primero horizontal por ciudad, luego vertical por sensibilidad.

paso, si el fragmento cumple completitud, reconstrucción y disjunción.

10. Paso 6: consultas globales con vistas

Cuando alguien pregunta “¿cuánto vendimos en total el 2 de marzo?”, la consulta no se puede resolver en un solo nodo. Lo que se hace es crear una *vista global*: una consulta con nombre que combina los tres fragmentos horizontales de pedidos.

Para que la práctica se pueda hacer sin herramientas adicionales, la vista global la simularemos en un cuarto script que se apoya en el módulo `postgres_fdw` de PostgreSQL, que es gratuito y viene incluido con la instalación estándar [4]. Este módulo permite que un nodo lea tablas de otro nodo como si fueran locales. El Listado 11 arma el ejemplo desde `pg-campus`.

```

1 CREATE EXTENSION IF NOT EXISTS postgres_fdw;
2
3 CREATE SERVER srv_babahoyo
4   FOREIGN DATA WRAPPER postgres_fdw
5   OPTIONS (host 'pg-babahoyo', dbname 'cafeteria', port '5432');
6
7 CREATE SERVER srv_ventanas
8   FOREIGN DATA WRAPPER postgres_fdw
9   OPTIONS (host 'pg-ventanas', dbname 'cafeteria', port '5432');
10
11 CREATE USER MAPPING FOR admin SERVER srv_babahoyo
12   OPTIONS (user 'admin', password 'admin123');
13 CREATE USER MAPPING FOR admin SERVER srv_ventanas
14   OPTIONS (user 'admin', password 'admin123');
15
16 CREATE FOREIGN TABLE pedidos_babahoyo (
17   pedido_id INTEGER, cliente_id INTEGER, producto_id INTEGER,
18   fecha DATE, monto NUMERIC(8,2), sede VARCHAR(20)
19 ) SERVER srv_babahoyo OPTIONS (table_name 'pedidos');
20
21 CREATE FOREIGN TABLE pedidos_ventanas (
22   pedido_id INTEGER, cliente_id INTEGER, producto_id INTEGER,
23   fecha DATE, monto NUMERIC(8,2), sede VARCHAR(20)
24 ) SERVER srv_ventanas OPTIONS (table_name 'pedidos');
25
26 -- Vista global reconstruida a partir de los tres fragmentos horizontales.
27 CREATE VIEW pedidos_global AS
28   SELECT * FROM pedidos           -- fragmento local (Campus)
29   UNION ALL
30   SELECT * FROM pedidos_babahoyo -- fragmento remoto
31   UNION ALL
32   SELECT * FROM pedidos_ventanas; -- fragmento remoto
33
34 -- Ahora podemos consultar como si estuviera todo en un solo lugar:
35 SELECT sede, SUM(monto) AS total
36 FROM pedidos_global
37 WHERE fecha = '2026-03-02'
38 GROUP BY sede;
39

```

Listing 11: 06_vistas_globales.sql — consulta global sobre los tres fragmentos horizontales usando `postgres_fdw`.

Fíjate en la esencia: la vista `pedidos_global` *materializa* la regla de reconstrucción horizontal usando `UNION ALL`. En el caso vertical, la reconstrucción se hace con `JOIN` por `cliente_id`, como muestra la línea ?? del Listado 12.

```

1 CREATE VIEW clientes_global AS
2   SELECT p.cliente_id, p.nombre, p.ciudad, c.email, c.telefono
3   FROM clientes_publicos p                      (*@\label{lst:vertical-reconstr}@*)
4   JOIN clientes_contacto_remota c USING (cliente_id);
5

```

Listing 12: Reconstrucción vertical de clientes.

11. Paso 7: verificación de las tres condiciones

Antes de dar por buena tu fragmentación, tienes que probar que cumple las tres reglas. El Listado 13 contiene tres consultas de comprobación. Ejecútalas y anota los resultados en tu informe.

```

1  -- 1) COMPLETITUD: la vista global debe tener el mismo numero
2  -- de filas que la tabla centralizada de referencia.
3  SELECT COUNT(*) AS filas_globales FROM pedidos_global;
4  -- Se compara con el conteo original en el nodo central de referencia.
5
6  -- 2) RECONSTRUCCION: una consulta general contra la vista global
7  -- debe dar el mismo resultado que la misma consulta contra la BD central.
8  SELECT sede, SUM(monto) FROM pedidos_global GROUP BY sede;
9
10 -- 3) DISJUNCION horizontal: ningun pedido debe aparecer en dos nodos.
11 SELECT pedido_id, COUNT(*) AS veces
12 FROM pedidos_global
13 GROUP BY pedido_id
14 HAVING COUNT(*) > 1;  -- debe devolver CERO filas.
15

```

Listing 13: 07_verificacion.sql — comprobación de completitud, reconstrucción y disjunción.

12. Entregables de la práctica

Cada equipo debe subir a su repositorio de GitHub una carpeta **practica-fragmentacion/** que contenga: el **docker-compose.yml**, los siete scripts SQL, un **README.md** con instrucciones claras de ejecución, y un documento **informe.pdf** de *cuatro a seis páginas* elaborado en **L^AT_EX**. El informe debe incluir la Figura 2 adaptada a los datos reales del equipo, capturas de **psql** mostrando el conteo de filas en cada nodo, y una reflexión de al menos una página sobre las ventajas y las dificultades observadas.

La rúbrica de evaluación es la de la Tabla 2. La calificación final se calcula con la fórmula estándar de la asignatura:

$$\text{Nota}_{/10} = \left[\frac{\sum_i w_i \cdot n_i}{4} \right] \cdot 10, \quad \sum_i w_i = 1,0 \quad (6)$$

donde w_i es el peso del criterio y $n_i \in \{0, 1, 2, 3, 4\}$ es el nivel alcanzado. Un criterio ausente puntúa CERO, sin excepción.

Criterio	Indicador clave (nivel 4)	Peso
C1. Entorno operativo	Los tres contenedores levantan con un solo <code>docker compose up</code> sin errores.	10 %
C2. Esquema centralizado	Se ejecuta el script <code>01_esquema_central.sql</code> sin fallos y respeta claves primarias y foráneas.	10 %
C3. Fragmentación horizontal	Los tres nodos contienen los subconjuntos correctos de <code>pedidos</code> .	15 %
C4. Fragmentación vertical	Los fragmentos <code>clientes_publicos</code> y <code>clientes_contacto</code> tienen la PK replicada y las columnas correctas.	15 %
C5. Fragmentación mixta	Se aplica correctamente la Tabla 1 y se justifica en el informe.	10 %
C6. Vistas globales con <code>postgres_fdw</code>	<code>pedidos_global</code> y <code>clientes_global</code> devuelven los mismos datos que la BD centralizada.	15 %
C7. Verificación	Las tres consultas del Listado 13 arrojan valores esperados y se documentan.	10 %
C8. Informe L ^A T _E X y reflexión	Cuatro a seis páginas, con figuras propias, referencias IEEE y una reflexión honesta.	15 %

Tabla 2: Rúbrica de evaluación de la práctica. Aplica la fórmula (6).

13. Errores comunes y cómo resolverlos

- **El puerto 5433, 5434 o 5435 está ocupado.** Cámbialo por otro libre en el `docker-compose.yml` y vuelve a levantar el sistema.
- **`docker exec` responde “No such container”.** El contenedor no se levantó. Ejecuta `docker compose ps` y revisa el estado; si dice `Exit`, mira los logs con `docker compose logs pg-campus`.
- **`postgres_fdw` no ve el nodo remoto.** Los tres contenedores deben estar en la misma red Docker. Con Docker Compose eso ocurre automáticamente porque comparten la red por defecto del proyecto.
- **Aparece un pedido en dos nodos.** Falló la disjunción. Revisa tus `INSERT`: cada `pedido_id` debe estar en un solo nodo.
- **La suma total no cuadra con la referencia.** Falló la completitud. alguna fila no fue insertada en ningún nodo o se insertó con un valor equivocado.

14. Preguntas para pensar

Estas preguntas no requieren código; se responden en el informe.

1. Si un día se abre una nueva sede en El Empalme, ¿qué debe cambiar en tu diseño horizontal? ¿Cuánto trabajo cuesta ese cambio?
2. ¿En qué situaciones la fragmentación vertical mejora la seguridad? Da un ejemplo distinto al de esta guía.
3. ¿Qué ventaja da que la clave primaria se replique en la fragmentación vertical? ¿Qué pasaría si no la replicáramos?
4. ¿Por qué la fragmentación horizontal por sede es una elección razonable para `pedidos`? ¿Habría sido mejor fragmentar por `producto_id`? Justifica.
5. Si tuvieras que soportar tolerancia a fallos, ¿qué agregarías a este diseño? Piensa en el concepto de *replicación* [1].

Referencias

- [1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Cham, Switzerland: Springer, 4 ed., 2020.
- [2] S. Ceri, S. B. Navathe, and G. Wiederhold, “Distribution design of logical database schemas,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 4, pp. 487–504, 1983.
- [3] S. B. Navathe, S. Ceri, G. Wiederhold, and J. Dou, “Vertical partitioning algorithms for database design,” *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 680–710, 1984.
- [4] PostgreSQL Global Development Group, “PostgreSQL 16 documentation — table partitioning.” Online documentation, 2024.